
Reinforcement Learning – Proximal Policy Optimization (PPO)

Coen Büller (s2631415)

Abstract

This report investigates Proximal Policy Optimization (PPO-Clip) applied to the CartPole-v1 environment. We conduct an hyperparameter optimization study over the number of sampled trajectories N , the GAE parameter λ and the clipping threshold ϵ , and compare the best-performing PPO configuration against A2C, REINFORCE, and DQN. PPO with optimized hyperparameters achieves the fastest convergence and highest asymptotic return, demonstrating the effectiveness of the clipped surrogate objective in stabilizing policy gradient training. However, performance is highly sensitive to hyperparameter choices, and CartPole’s simplicity limits the extent to which differences between methods can be observed.

1. Introduction

Policy-based methods, like REINFORCE (Williams, 1992), have many advantages compared to value-based methods, such as being able to learn stochastic policies, being effective in large or continuous action spaces, and exploring naturally due to the parameterization of the policy (π_θ) (Plaat, 2022). They do so by optimizing their parameters following:

$$\nabla_\theta J(\theta) = \mathbb{E}_{a_t \sim \pi_\theta} \left[\hat{Q}(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t | s_t) \right] \quad (1)$$

To estimate the future return $\hat{Q}(s, a)$ policy-based method often requires sampling a full trajectory (τ), also called Monte-Carlo sampling, before updating its network parameters. This results in low bias, but also low convergence rates, convergence to local minima, and high variance (Plaat, 2022).

One extremely popular method is the Actor-Critic (AC) method (Barto et al., 1983). This combines the a policy-based method (actor), while reducing variance by adding a value-based network (critic) to estimate the expected return.

$$\hat{Q}_n(s_t, a_t) = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V_\phi(s_{t+n}) \quad (2)$$

Where γ is the discount factor which reduces the influence of future rewards. $\hat{Q}(s_t, a_t)$ in equation 1 can be substituted by 2 to gain the objective gradient of the policy-network. The value network can be optimized minimizing the following loss function.

$$\mathcal{L}(\phi | s_t, a_t) = \left(\hat{Q}_n(s_t, a_t) - V_\phi \right)^2 \quad (3)$$

To further decrease the variance of the AC network most will use the advantage $\hat{A}_\phi(s_t, a_t)$ (Eq. 4) instead of $\hat{Q}(s_t, a_t)$, which is called the Advantage Actor-Critic (A2C).

$$\hat{A}_\phi(s_t, a_t) = \hat{Q}_n(s_t, a_t) - V_\phi(s_t) \quad (4)$$

The A2C method reduces the variance significantly, and has gained a lot of popularity due to that. However, many other definitions for the advantage have been developed. One of these is the Generalized Advantage Estimation (GAE) (Schulman et al., 2018).

$$\hat{A}^{\text{GAE}(\gamma, \lambda)}(s_t) = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^{V_\phi} \quad (5)$$

$$\delta_t^{V_\phi} = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \quad (6)$$

$\delta_t^{V_\phi}$ is the temporal difference error of the network at step t . The parameter λ is a parameter that lets you control the bias-variance trade off. If $\lambda = 0$, GAE simply becomes the $\delta_t^{V_\phi}$ resulting in high bias, and by setting $\lambda = 1$, the GAE will look infinitely far ahead making it equal to MC sampling which converges to the advantage in equation 4 which results in high variance.

Several other attempts have been made to further reduce the variance. One of these attempts uses Trust Region Policy Optimization (TRPO), and its successor PPO. This report aims at exploring the capabilities of PPO in the Gymnasium CartPole-v1 environment (Towers et al., 2024), and compare the results to A2C, REINFORCE, and DQN from assignment 3 and 2. The theoretical foundation for TRPO, and its successor PPO will be discussed in section 2. Section 3 will go into the experimental setup and the hyperparameter selection for the PPO network. Section 4 will discuss the results, and section 5 and 6 contain the discussion and conclusion.

2. Theory

2.1. Trust Region Policy Optimization

Actor-Critic methods can still show too large variance in training. Especially when the agent is in a high return state, a single bad gradient update can push the agent into a low average return state (Hsu et al., 2020). TRPO tries to limit this by constraining the size of the policy update, which allows for the largest possible step to take without collapsing the performance (Piaat, 2022). More specifically, TRPO optimizes an objective function subject to a constraint on the objective function (Schulman et al., 2017).

$$\arg \max_{\theta} \mathbb{E}_t \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \cdot \hat{A}_t \right] \quad (7)$$

$$\text{subject to } \mathbb{E}_t [D_{KL}(\pi_{\theta_{old}}(\cdot|s_t), \pi_{\theta}(\cdot|s_t))] \leq \delta \quad (8)$$

The objective measures how well the new policy π_{θ} performs given a path generated by the old policy $\pi_{\theta_{old}}$, while being constrained by the KL -divergence between policies (Achiam, 2018). However, implementing this is quite difficult and requires a second order derivative (Achiam, 2018; Piaat, 2022). This led to Proximal Policy Optimization (PPO) (Schulman et al., 2017), which has much of the benefits of TRPO, but much simpler to implement (Piaat, 2022).

2.2. Proximal Policy Optimization

There are two PPO implementations, PPO-Penalty, and PPO-Clip (Achiam, 2018). Here we'll focus on PPO-Clip which updates policies through

$$\theta_{k+1} = \arg \max_{\theta} \mathbb{E} [\mathcal{L}(a_t, s_t, \theta_k, \theta)] \quad (9)$$

$$\mathcal{L}(s_t, a_t, \theta_{old}, \theta) = \min \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \hat{A}_t^{\pi_{\theta_{old}}}, g(\epsilon, \hat{A}_t^{\pi_{\theta_{old}}}) \right) \quad (10)$$

$$g(\epsilon, \hat{A}_t^{\pi_{\theta_{old}}}) = \text{clip} \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t^{\pi_{\theta_{old}}} \quad (11)$$

$\hat{A}_t^{\pi_{\theta_{old}}}$ is any advantage of action $a_t \sim \pi_{\theta_{old}}$ in state s_t . If the advantage is positive, the objective will increase if the action becomes more likely (Achiam, 2018). However, the clipping removes the incentive for the ratio to exceed $(1 + \epsilon)$, so the new policy does not deviate too far from the old policy. Vice versa for negative advantage. PPO-Clip algorithm is shown in Algorithm (1).

In Alg. 1 $\mathcal{L}^s$ is the smooth L_1 loss. Compared to A2C, PPO-Clip replaces the direct gradient step with a clipped surro-

Algorithm 1 PPO-Clip

```

1: Initialize policy parameters  $\theta$  and value parameters  $\phi$ 
2: repeat
3:   Collect  $N$  trajectories using  $\pi_{\theta_{old}}$ 
4:   Compute returns  $\hat{Q}_t$  and advantages  $\hat{A}_t$  (e.g. GAE)
5:   for  $k = 1, \dots, K$  do
6:      $r_t(\theta) \leftarrow \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ 
7:      $L^{\text{clip}} \leftarrow \mathbb{E}_t \left[ \min \left( r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$ 
8:     Update  $\theta$  by maximizing  $L^{\text{clip}}$ 
9:     Update  $\phi$  by minimizing  $\mathcal{L}^s(\hat{Q}_t, V_{\phi}(s_t))$ 
10:  end for
11: until convergence

```

gate objective. Important to note is that the algorithm described in Alg. 1 does K gradient updates over the sampled trajectories. This increases the sample efficiency greatly. Further note is that by setting $K = 1$, the algorithm effectively reduces back to A2C. Since the updates occur at the end of the loops, and the updating of the old policy at the beginning, the ratios will always be 1 for the first parameter update of the networks.

3. Experiments

Hyperparameter Optimization Study The network contains multiple hyperparameters that can influence the agent's performance. Compared to the previous assignment, several new parameters were introduced: K , ϵ , λ , and N . To determine which hyperparameter settings perform best, we individually evaluate $\epsilon \in \{0.05, 0.1, 0.2\}$, $N \in \{1, 5, 15\}$, and $\lambda \in \{0.7, 0.9, 0.99\}$. The parameter K is fixed at 10. We chose not to perform experiments on K , as this would significantly increase the computational cost, while larger values are generally expected to improve performance through increased sample efficiency. For each of these hyperparameters, the best performing setting will be chosen for the final PPO configuration. Furthermore for the baseline settings of the agent we'll use $\gamma = 0.99$, $\lambda = 0.9$, $n = 10$ for $\hat{Q}_n(s_t, a_t)$, $\epsilon = 0.2$, and $K = 10$.

Comparison To evaluate PPO, we compare the best-performing PPO configuration against A2C and REINFORCE from Assignment 3, and the DQN baseline from Assignment 2. All agents use a policy network with 4 input neurons, two hidden layers of 256 neurons with ReLU activations, and a 2-neuron Softmax output layer. The value networks use the same architecture, but differ in the final layer. PPO uses a single output neuron, while A2C uses two output neurons without activation.

All networks are optimized using AdamW with learning rates of $5 \cdot 10^{-5}$ for the policy networks, and $1 \cdot 10^{-4}$ for

the value network. PPO uses GAE (Eq. 5) for the policy update and the advantage function (Eq. 3, with $n = 10$) for the value update with. PPO additionally applies gradient clipping at 0.5 and 1.0 for the policy and value networks respectively. Finally, A2C, and PPO standardize advantages, A2C also standardizes the targets for value network, and both A2C, REINFORCE and PPO divide the policy loss by the length of the trajectory.

Environment We evaluate PPO on CartPole-v1, where an agent must balance an inverted pendulum by moving a cart left or right. The environment has a 4-dimensional continuous state space, a discrete 2-action space, and provides a reward of +1 per timestep until failure or a maximum episode length of 500 steps.

All experiments will be run for 5 independent iterations, 250,000 steps, with 10 evaluation episodes every 5000 steps from which we will record the average. The averages over the 5 iterations will be compared.

4. Results

The results for the hyperparameter optimization are shown in Figure 1, displaying the mean greedy return over environment steps for PPO-Clip under different hyperparameter settings. The left figure varies N , the middle varies λ , and the right varies ϵ . For all hyperparameters there is a significant difference in performance between settings, where larger N and smaller ϵ most reliably converge to the maximum average return within the 250,000 step budget. The improved convergence stability with larger N is expected, as collecting multiple trajectories reduces the variance of the gradient estimate $\nabla_{\theta} J(\theta)$, making it more accurately reflect the true gradient of the objective. Conversely, $N = 1$ yields a high-variance gradient estimate, which can cause erratic updates and unstable training. It is interesting to observe that $N = 5$ performs on average worse than $N = 1$. Since all experiments are run for a fixed number of total environment steps, increasing N directly reduces the number of policy updates performed during training, suggesting that this reduction in updates outweighs the benefit of a more accurate gradient estimate for $N = 5$. It is furthermore clear that $\lambda = 0.99$ consistently achieves the highest average return. This is somewhat surprising, as $\lambda \approx 1$ causes GAE to approach the Monte Carlo advantage estimate (Eq. 4 with $n = |\tau|$), which is known to have high variance. However, advantage standardization partially mitigates this, and more importantly, this result suggests that the bias introduced by lower λ values is more detrimental than the increased variance. This is consistent with $\lambda = 0.7$ producing systematically biased advantage estimates, causing consistently incorrect policy gradient updates and poor average return. Setting $\epsilon = 0.05$ also achieves the highest average return, demonstrating the effectiveness of PPO-Clip. Larger val-

ues of ϵ suffer from higher instability, where a single bad update can push the agent into a low-return state, as clearly visible for $\epsilon = 0.1$ and $\epsilon = 0.2$. Based on these results, the final PPO configuration is set to $N = 15$, $\lambda = 0.99$, and $\epsilon = 0.05$

As shown in Figure 2, PPO-Clip with optimized hyperparameters clearly outperforms all other evaluated methods, achieving both the fastest convergence rate and the highest asymptotic return, approaching the environment’s maximum score within the 2.5×10^5 step budget. This performance highlights the critical importance of hyperparameter optimization. However, it is important to note that A2C and REINFORCE were evaluated using baseline configurations without a corresponding tuning process, making this a fundamentally unfair comparison. Despite this limitation, PPO exhibits remarkably stable training trajectories with minimal performance degradation. Figure 1 already demonstrated that the effectiveness of clipping the gradients. When this is combined with good other hyperparameter settings, it prevents the policy collapse which is observed within the other agents.

5. Discussion

PPO has shown strong results in terms of convergence speed and stability, especially when configured with the proper hyperparameter settings (Fig. 2). This demonstrates that there is no single mechanism that can completely solve the inherent instability issues of reinforcement learning. Rather, it requires the cooperation of multiple intersecting techniques to make an algorithm successful. For example, we observed in Figure 1 that a clipping threshold of $\epsilon = 0.05$ achieves the highest average return. This does, however, mean that the convergence rate can initially be lower as the clipping is so strict that some iterations get stuck in local optima, as suggested by the high variance in the early timesteps. It was also observed that increasing the batch size has a very strong positive effect on performance. While a larger batch size typically decreases sample efficiency by requiring more environment steps per gradient update, these negative effects can be mitigated by increasing the number of optimization epochs K to reuse the collected data. Ultimately, by increasing both K and the batch size, sufficient stability is established, which in turn allows for a larger ϵ to achieve a faster overall convergence rate.

Furthermore, this also means that the comparison to A2C, REINFORCE in section 3 cannot be compared directly as they were tested on baseline hyperparameter settings. Figure 1 has shown the importance of hyperparameter tuning to achieve a high and stable average return. However, a key issue observed in Assignments 2 and 3 was training instability. This issue was partially mitigated by introducing clipping and applying hyperparameter tuning, which

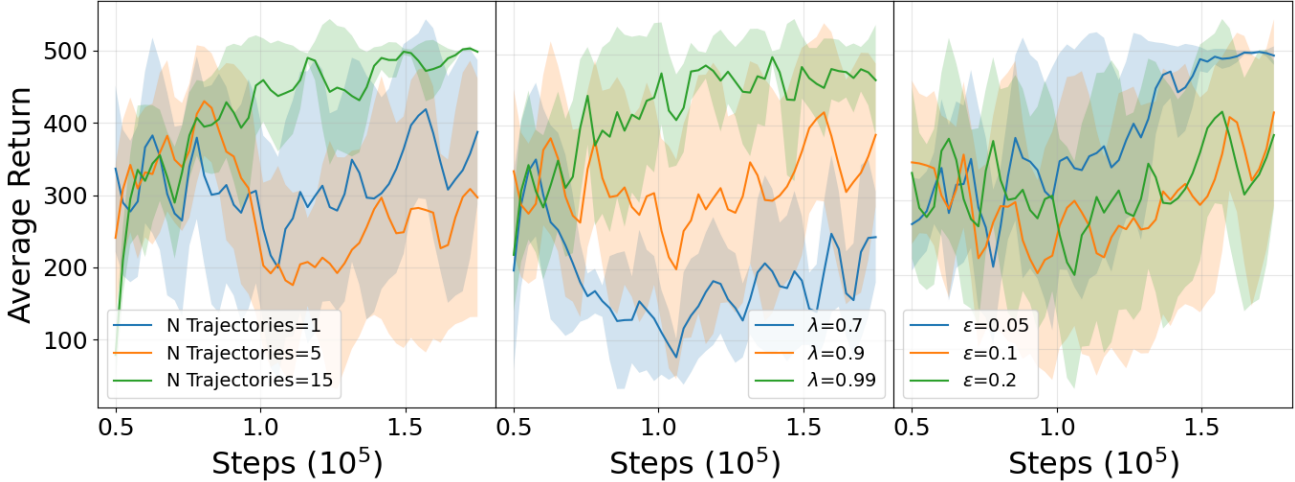


Figure 1. Mean greedy return over environment steps for PPO-Clip under different hyperparameter settings. Shaded areas represent the standard deviations across independent runs. The left figure varies the number of sampled trajectories N , the middle figure varies λ , and the right figure varies ϵ . All hyperparameters have a noticeable effect on performance. In particular, larger values of N and smaller values of ϵ result in more stable training and more consistent convergence toward the maximum average return of 500 within 250,000 environment steps. A Savitzky–Golay filter with a window size of 5 and polynomial degree 2 is applied to smooth the curves.

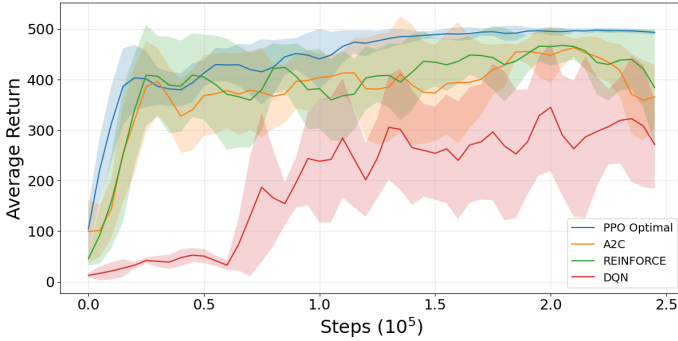


Figure 2. Mean evaluation return over environment steps for PPO-Clip (optimized), A2C, REINFORCE, and DQN. Shaded areas represent the standard deviations across independent runs. PPO shows the fastest convergence and highest training stability, while DQN struggles to converge within the 2.5×10^5 step budget. Curves are smoothed using a Savitzky–Golay filter (window size 5, polynomial degree 2).

improved overall training stability.

CartPole-v1 is a relatively simple environment for the agent to solve. To further test the capabilities and limitations of PPO, we will need to evaluate it in a more challenging environment. As noted in (Hsu et al., 2020), PPO-Clip attempts to limit deviations from the old policy by clipping the probability ratio within the surrogate objective function. However, the actual gradient steps taken in parameter space are not inherently constrained by PPO-Clip, which can still result in an unfavorable parameter update leading to policy collapse. According to the authors, PPO-Penalty achieves

better average returns in environments with a large or continuous action space. It would be interesting to compare both methods further, but it will be left for future work.

Furthermore, PPO-Clip tries to overcome policy collapse by approximating trust regions. Another possible plan of attack is adding more exploration during training. As the agent explores more during training, it explores a larger portion of the search space and can avoid policy collapse (Plaat, 2022). One good way could be by adding an entropy penalty to the loss function which stimulates exploration, also called Soft Actor-Critic (SAC) (Haarnoja et al., 2018). However, this will also be left for further research.

6. Conclusion

In this work, we implemented PPO-Clip and evaluated it on the CartPole-v1 environment, comparing it against A2C, REINFORCE, and DQN. An hyperparameter optimization study identified $N = 15$, $\lambda = 0.99$, and $\epsilon = 0.05$ as the strongest configuration, highlighting the sensitivity of PPO to its hyperparameters. With optimized settings, PPO achieved the fastest convergence and highest asymptotic return among all evaluated methods. These results confirm that PPO’s clipped surrogate objective effectively prevents destructive policy updates, though its performance remains contingent on careful hyperparameter selection. Future work should evaluate PPO in more complex environments, compare PPO-Clip against PPO-Penalty in continuous action spaces and explore entropy regularization on the policy loss for improved exploration.

References

- Achiam, J. Spinning up in deep reinforcement learning. <https://github.com/openai/spinningup>, 2018.
- Barto, A. G., Sutton, R. S., and Anderson, C. W. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13(5):834–846, 1983. doi: 10.1109/TSMC.1983.6313077.
- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018. URL <https://arxiv.org/abs/1801.01290>.
- Hsu, C. C.-Y., Mendler-Dünner, C., and Hardt, M. Revisiting design choices in proximal policy optimization, 2020. URL <https://arxiv.org/abs/2009.10897>.
- Plaat, A. Deep reinforcement learning: A textbook. *arXiv preprint arXiv:2201.02135*, 2022. doi: 10.48550/arXiv.2201.02135.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation, 2018. URL <https://arxiv.org/abs/1506.02438>.
- Towers, M. et al. Gymnasium: A standard interface for reinforcement learning environments. https://gymnasium.farama.org/environments/classic_control/cart_pole/, 2024. Accessed: 2026-04-01.
- Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. 8(3–4), 1992. ISSN 0885-6125. doi: 10.1007/BF00992696. URL <https://doi.org/10.1007/BF00992696>.